MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

COURSE PROJECT REPORT

CSD201 – DATA STRUCTURES AND ALGORITHMS (JAVA)

TOPIC: LARGE-SCALE INTEGER KEY SORTING

TEXTBOOK PARADIGMS AND PATTERN-DEFEATING QUICKSORT (PDQSORT)

Academic Supervisor: Department of Computer Science
Class Code: SE20C
Academic Term: Fall 2026
Project Team: GROUP 6

GROUP MEMBERS AND PAIR ALLOCATION:

No.	Full Name	Student ID	Pair	Core Technical Responsibilities
1	Nguyễn Thành Danh	QE200138	Pair 1	Quick Sort, Killer Adversary Analysis
2	Võ Xuân Long	QE200140	Pair 1	Merge Sort, Data Generator D1–D5
3	Phan Văn Bảo	QE200113	Pair 2	Heap Sort, Radix Sort (Lead RQ2 Analysis)
4	Nguyễn Nhật Tường Vy	QE200093	Pair 2	BenchmarkRunner, MXBean Metrics & CSV
5	Trần Hoàng Thịnh	QE200056	Pair 3	Core pdqsort, Introsort Heap Fallback
6	Nguyễn Đồng Tấn Phước	QE200014	Pair 3	Variants noA/noB, Verification Suite (Lead RQ3)

Khảo sát thực nghiệm thuật toán sắp xếp khóa số nguyên quy mô lớn: Thuật toán giáo trình và Pattern-Defeating Quicksort (pdqsort)

Nguyễn Thành Danh*, Võ Xuân Long*, Phan Văn Bảo†, Nguyễn Nhật Tường Vy†, Trần Hoàng Thịnh‡, Nguyễn Đồng
Khoa Công nghệ Thông tin, Trường Đại học FPT
Email: {DanhNTQE200138, LongVXQE200140, BaoPVQE200113, VyNNTQE200093, ThinhTHQE200056, PhuocNDTQE200014
*Cặp 1: Quick Sort, Merge Sort, Generator †Cặp 2: Heap Sort, Radix Sort, Runner ‡Cặp 3: pdqsort, Ablation

***Tóm tắt nội dung*—Việc sắp xếp hiệu quả các mảng khóa số nguyên quy mô lớn đóng vai trò nền tảng trong các hệ thống đối soát dữ liệu và xử lý giao dịch thông lượng cao. Trong khi các môi trường thực thi ngôn ngữ hiện đại (như Go, Rust) đã chuyển đổi sang các thuật toán lai thích ứng như Pattern-Defeating Quicksort (pdqsort), giáo trình đại học chuẩn chủ yếu nhấn mạnh các thuật toán kinh điển gồm Quick Sort, Merge Sort, Heap Sort và Radix Sort. Nghiên cứu này cung cấp một so sánh thực nghiệm có kiểm soát chặt chẽ trên năm phân phối dữ liệu đặc thù (D_1–D_5) với quy mô mở rộng từ 10^5 đến 2×10^7 phần tử. Chúng tôi đánh giá toàn diện thời gian thực thi, dung lượng bộ nhớ đỉnh, số lượng thao tác cơ bản và độ sâu đệ quy theo giao thức đo chuẩn hội nghị IEEE. Đồng thời, nghiên cứu khảo sát pdqsort cùng các biến thể triệt tiêu cơ chế nhận diện mẫu (noA và noB). Kết quả cho thấy Radix Sort đạt tốc độ tăng trưởng dưới tuyến tính ổn định trên mọi phân phối đầu vào, trong khi Heap Sort bảo đảm nghiêm ngặt bộ nhớ phụ $O(1)$ nhưng chịu suy giảm hiệu năng do tổn thất truy cập bộ nhớ đệm phi cục bộ. Đáng chú ý, pdqsort thích nghi với các mảng đã có thứ tự trong thời gian gần như tuyến tính ($O(N)$) và loại bỏ hoàn toàn sự suy thoái khi gặp khóa trùng lặp nhờ phân hoạch ba chiều, cung cấp các tiêu chí điều kiện rõ ràng để lựa chọn thuật toán trong các hệ thống thực tế.**

***Index Terms*—Sắp xếp số nguyên, Quick Sort, Radix Sort, Heap Sort, pdqsort, đo đạc thuật toán, đánh giá thực nghiệm.**

I. Giới thiệu (Introduction)

Trong các hệ thống xử lý giao dịch tài chính và ngân hàng, việc sắp xếp hàng chục triệu mã giao dịch số nguyên 32-bit ($0 \le x \le 2^{31} - 1$) diễn ra thường nhật. Dữ liệu thực tế hiếm khi phân phối hoàn toàn ngẫu nhiên: nhiều lô dữ liệu gần như đã có thứ tự theo mốc thời gian, hoặc chứa mật độ phần tử trùng lặp rất lớn.

Các giáo trình cấu trúc dữ liệu và giải thuật chuẩn (CLO6) thường tập trung vào bốn thuật toán kinh điển: Quick Sort, Merge Sort, Heap Sort và Radix Sort. Tuy nhiên, các ngôn ngữ lập trình hiện đại như Go và Rust đã chuyển sang dùng Pattern-Defeating Quicksort (pdqsort) do Orson Peters đề xuất [4]. pdqsort nổi tiếng với khả năng tự nhận diện các dạng mẫu dữ liệu có thứ tự và khóa trùng lặp, đồng thời duy trì bảo đảm thời gian trường hợp xấu nhất $O(N \log N)$.

Để cung cấp cơ sở khoa học định lượng cho việc lựa chọn giải thuật trong các hệ thống thực tế, nghiên cứu này giải quyết ba câu hỏi nghiên cứu cốt lõi:

- **RQ1 (Quy mô dữ liệu):** Bốn thuật toán giáo trình khác nhau như thế nào về thời gian thực thi, đỉnh bộ nhớ phụ và số thao tác cơ bản khi quy mô N tăng từ 10^5 đến 2×10^7?
- **RQ2 (Dạng dữ liệu):** Mức độ có thứ tự ($D_1 \to D_4$) và mật độ trùng khóa (D_5) làm thay đổi thứ hạng tương đối của bốn thuật toán giáo trình như thế nào?
- **RQ3 (Đóng góp của pdqsort):** pdqsort cải thiện bao nhiêu so với Quick Sort giáo trình, và các cơ chế cốt lõi (Cơ chế A: phát hiện đoạn đã sắp, Cơ chế B: gom khóa trùng) đóng góp bao nhiêu thông qua thí nghiệm triệt tiêu (`noA`, `noB`)?

II. Cơ sở lý thuyết (Background)

A. Các thuật toán dựa trên so sánh khóa

Quick Sort do Hoare đề xuất [1] áp dụng kỹ thuật chia để trị. Với phân hoạch Hoare hai con trỏ và chọn chốt trung vị của 3 (median-of-three), thuật toán có độ phức tạp trung bình $O(N \log N)$ nhưng vẫn đối mặt với nguy cơ $O(N^2)$ và tràn ngăn xếp (StackOverflowError) khi gặp các mảng sát thủ (killer adversary) [5].

Merge Sort bảo đảm thời gian $O(N \log N)$ và tính ổn định [7], song đòi hỏi mảng phụ kích thước $O(N)$, gây áp lực đáng kể lên hệ thống bộ nhớ đệm và phân mảnh heap.

Heap Sort do Williams đề xuất [2] sử dụng Max-Heap tại chỗ, bảo đảm thời gian trường hợp xấu nhất $O(N \log N)$ và bộ nhớ phụ nghiêm ngặt $O(1)$. Tuy nhiên, do truy cập nhảy bước nhảy xa trên cây nhị phân, Heap Sort chịu tổn thất lớn về tính cục bộ của bộ nhớ cache (cache locality).

B. Thuật toán phân phối phi so sánh

Radix Sort (LSD) [7] bóc tách khóa nguyên theo từng nhóm bit (cơ số 256, 8-bit/lượt). Thuật toán không so sánh giá trị khóa mà phân phối dựa trên Counting Sort, đạt độ phức tạp lý thuyết $O(k \cdot N)$ với $k = 4$ lượt cho số nguyên 32-bit.

Đồ án môn học CSD201 FA26 – Đề tài 1. Hướng dẫn bởi Bộ môn Khoa học Máy tính, Đại học FPT.

C. Pattern-Defeating Quicksort (pdqsort)

pdqsort [4] kết hợp nhiều kỹ thuật tinh tế: 1) Ngưỡng cơ sở chuyển sang Insertion Sort khi đoạn con $N < 24$. 2) Phân hoạch thích nghi với Tukey's Ninther [3] khi $N > 128$. 3) **Cơ chế A:** Thử Partial Insertion Sort trên hai nửa nếu phân hoạch cân bằng và không có hoán đổi, đạt $O(N)$ trên dữ liệu tăng dần. 4) **Cơ chế B:** Gom các khóa bằng pivot sang trái khi pivot bằng phần tử liền trước, loại trừ suy thoái trên mảng trùng lặp. 5) Chống suy thoái (Introsort limit): Đếm số lần phân hoạch lệch; nếu vượt $\lfloor \log_2 N \rfloor$ thì lập tức chuyển sang Heap Sort trên đoạn đang xét.

III. Phương pháp cài đặt (Methods)

Mọi thuật toán được hiện thực bằng Java 17 trên mảng thuần nguyên bản `int[]`, không dùng cấu trúc bao gói (wrapper) hay Comparator.

A. Cài đặt Quick Sort phân hoạch Hoare và Median-of-Three (Cặp 1)

Quick Sort được cài đặt chuẩn mực theo phân hoạch Hoare hai đầu: hai con trỏ i và j chạy hội tụ từ hai phía và dừng lại ngay khi gặp khóa bằng pivot ($a[i] \geq$ pivot và $a[j] \leq$ pivot). Pivot được lựa chọn là trung vị của ba phần tử đầu, giữa và cuối đoạn (`medianOfThree`: lo, mid, hi). Thuật toán đệ quy cả hai phía, không chuyển sang Insertion Sort và không sử dụng phân hoạch Lomuto. Ở lượt đo `-count`, hàm `sortWithCount` đếm chính xác số phép so sánh khóa và độ sâu đệ quy tối đa (`max_depth`), đóng gói vào số nguyên 64-bit `long` (32 bit cao là `max_depth`, 32 bit thấp là `comparisons`).

B. Cài đặt Merge Sort Top-Down Tái Sử Dụng Mảng Phụ (Cặp 1)

Merge Sort được xây dựng theo mô hình Top-Down chia đôi đệ quy từ trên xuống. Để triệt tiêu overhead cấp phát bộ nhớ lặp lại, thuật toán chỉ cấp phát **duy nhất một lần** mảng phụ kích thước N (`int[] aux = new int[N]`) tại hàm gọi ban đầu và truyền tham chiếu xuyên suốt cây đệ quy. Quá trình trộn hai mảng con tuân thủ nghiêm ngặt quy định không tối ưu bỏ qua bước trộn (tuyệt đối không dùng `if (a[mid] <= a[mid+1]) return;`).

C. Cài đặt Heap Sort và Hàm Fallback cho pdqsort (Cặp 2)

Heap Sort được cài đặt max-heap tại chỗ bằng phương pháp dựng bottom-up từ chỉ số $(n/2 - 1)$ xuống 0. Để hỗ trợ pdqsort cứu nguy khi đệ quy lệch, mô-đun cung cấp hàm:

```
public static void sortRange(int[] a, int lo,
                             int hi)
```

Hàm này sắp xếp đoạn nửa mở $[lo, hi)$ mà không tác động đến các phần tử ngoài phạm vi, bảo đảm trường hợp xấu nhất của pdqsort không bao giờ vượt quá $O(N \log N)$.

D. Cài đặt LSD Radix Sort 4 Lượt Cơ Số 256 (Cặp 2)

Radix Sort xử lý đúng 4 lượt ứng với 4 byte của số nguyên không âm ($0 \leq x \leq 2^{31} - 1$). Nhóm tối ưu hóa bộ nhớ phụ bằng cách chỉ cấp phát **duy nhất 1 mảng phụ** `aux` kích thước N và 1 mảng đếm 256 phần tử. Nhờ số lượt chẵn (4 lượt), dữ liệu được tráo đổi luân phiên giữa `a` và `aux`, tự động kết thúc chính xác tại mảng gốc mà không tốn thao tác sao chép dư thừa.

E. Cài đặt pdqsort và Các Biến thể Triệt tiêu (Cặp 3)

pdqsort hiện thực đầy đủ thiết kế của Peters [4]:

- Ngưỡng cơ sở: Đoạn dưới 24 phần tử sắp xếp bằng Insertion Sort (sử dụng bản không kiểm tra biên `unguardedInsertionSort` cho các đoạn bên trong).
- Chọn pivot: Trung vị của 3 cho đoạn ≤ 128; đoạn trên 128 phần tử dùng Ninther của Tukey (trung vị của 3 trung vị).
- Phân hoạch phải (`partitionRight`): Đưa phần tử $<$ pivot sang trái, $\geq$ pivot sang phải; trả về cờ `alreadyPartitioned` nếu không phải tráo đổi phần tử nào.
- **Cơ chế A (Phát hiện đoạn đã sắp):** Khi phân hoạch cân bằng và đã phân hoạch sẵn, thực hiện Partial Insertion Sort trên cả hai nửa với giới hạn dịch tối đa 8 lần.
- **Cơ chế B (Gom khóa trùng):** Khi pivot bằng phần tử đứng liền trước đoạn, sử dụng phân hoạch trái (`partitionLeft`) gom toàn bộ khóa trùng sang trái rồi bỏ qua.
- **Chống suy thoái:** Đếm phân hoạch lệch (khi một nửa $< size/8$), hoán đổi phá mẫu (`breakPatterns`); nếu số lần lệch vượt $\lfloor \log_2 N \rfloor$ thì lập tức chuyển đoạn đó sang `HeapSort.sortRange`.

Hai biến thể triệt tiêu `noA` và `noB` được kích hoạt độc lập qua tham số dòng lệnh `-variant`.

F. Bộ Sinh Dữ Liệu và Ca Kiểm Thử Đối Kháng Killer Adversary

Bộ sinh `DataGenerator` cung cấp 5 họ dữ liệu chuẩn $D_1 \rightarrow D_5$ bằng `SplittableRandom(seed)`. Đặc biệt, nhóm đã hiện thực thuật toán sinh mảng sát thủ **Killer Adversary** theo phương pháp của M. D. McIlroy [6] và D. R. Musser [5]. Cơ chế này mô phỏng phản ứng của bộ so sánh trực tiếp với phân hoạch Hoare và pivot median-of-three, cố tình gán giá trị làm lệch cực đại mọi bước phân hoạch ($size - 1$ và 1), ép Quick Sort giáo trình phải chịu $O(N^2)$ so sánh và độ sâu đệ quy tới $N/2$.

G. Quy tắc đếm thao tác công bằng và Độ sâu đệ quy

Đề tài quy định rõ:

- Với Quick, Merge, Heap, pdqsort: đếm số phép so sánh khóa trực tiếp giữa hai phần tử dữ liệu.
- Với Radix Sort: do không so sánh khóa, đếm số lần đọc và ghi mảng dữ liệu. Mỗi lượt tốn N đọc (đếm tần suất) + N đọc và N ghi (phân phối) = $3N$. Qua 4 lượt, tổng số thao tác cố định là $12N$.

- Độ sâu đệ quy (`max_depth`): Ghi nhận mức sâu nhất của ngăn xếp gọi hàm ở lượt `-count`.

H. Bộ Kiểm Thử Tự Động (Verification Suite) và Oracle

Toàn bộ dự án được bảo vệ bằng bộ kiểm thử tự động gồm **42 ca kiểm thử JUnit 5** bao phủ 100% các mô-đun: 1) Đạt chuẩn 5 ca cố định theo đề bài ([7], [2, 1], [3, 3, 3], $[0, 2^{31} - 1, 0]$, $[5, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]$). 2) Kiểm thử 1.000 mảng ngẫu nhiên kích thước $0 \to 100$ thuộc $D_1 \to D_5$ so với oracle `Arrays.sort`. 3) Kiểm thử mảng lớn 10^5 phần tử: toàn bằng nhau, tăng dần, giảm dần. 4) Sau mỗi lượt đo thực nghiệm, mảng kết quả được kiểm tra khớp từng phần tử với `Arrays.sort` và tính mã băm đa thức 64-bit:

$$h = \sum_{i=0}^{N-1} 31^{N-1-i} \cdot a[i] \pmod{2^{64}} \tag{1}$$

để bảo đảm tính đúng đắn tuyệt đối.

IV. Thiết lập thực nghiệm (Experimental Setup)

- **Phần cứng và Môi trường:** Máy đo thử nghiệm trang bị CPU 8 lõi, 16 GB RAM, hệ điều hành Windows 11 64-bit, JDK 17 (HotSpot 64-bit Server VM) với cờ cấp phát heap cố định `-Xmx8g`.
- **Quy trình đo có kiểm soát:** Mỗi tổ hợp cấu hình được khởi chạy trong một tiến trình JVM độc lập qua `ProcessBuilder` với giới hạn thời gian (timeout) nghiêm ngặt 60 giây.
- **Làm nóng JIT (Mục 6.2):** Mỗi tiến trình chạy làm nóng 3 lần trên mảng ngẫu nhiên 50.000 phần tử trước khi kích hoạt bộ đếm thời gian.
- **Cô lập phép đo:** Thời gian được đo bằng `System.nanoTime()` bao bọc duy nhất hàm sắp xếp. Đỉnh bộ nhớ được ghi nhận qua `MemoryPoolMXBean` sau khi cưỡng bức `System.gc()`. Lượt đo số phép tính (`-count`) được tách riêng biệt thành một lượt chạy độc lập.
- **Bộ dữ liệu thử nghiệm (Bảng 4):** D_1 (Ngẫu nhiên đều), D_2 (Tăng dần), D_3 (Gần tăng dần, hoán đổi $N/1000$ cặp), D_4 (Giảm dần hoàn toàn), và D_5 (Trùng lặp với $K \in \{1000, 10, 2\}$).

V. Kết quả và Thảo luận (Results & Discussion)

A. RQ1: Hiệu năng khi quy mô N tăng trưởng (D_1)

Bảng I trình bày thời gian thực thi trung vị (median ms) và dung lượng bộ nhớ đỉnh khi quy mô dữ liệu N tăng dần từ 10^5 đến 2×10^7 trên phân phối ngẫu nhiên D_1.

Phân tích RQ1:

1) **Sự vượt trội của Radix Sort:** Radix Sort đạt tốc độ nhanh hơn Quick Sort từ 4 đến 6 lần trên quy mô lớn ($N = 2 \times 10^7$). Điều này phản ánh ưu thế tuyệt đối của thuật toán có độ phức tạp tuyến tính $O(4N)$ khi so với các thuật toán $O(N \log N)$.

2) **Hiệu ứng Cache của Heap Sort:** Mặc dù Heap Sort có độ phức tạp lý thuyết $O(N \log N)$ tương đương Quick Sort và Merge Sort, thời gian thực thi thực tế của nó chậm hơn gấp đôi khi $N \geq 10^7$. Nguyên nhân cốt lõi là do cấu trúc heap nhị phân phân bố dữ liệu theo bước nhảy cấp số nhân $(2i + 1, 2i + 2)$, gây ra tỷ lệ trượt bộ nhớ đệm (cache miss rate) rất cao trên không gian mảng vượt kích thước L3 Cache.

3) **Bộ nhớ phụ:** Merge Sort và Radix Sort yêu cầu cấp phát đúng mảng phụ $N \times 4$ byte (≈ 80 MB tại $N = 2 \times 10^7$), trong khi Heap Sort duy trì bộ nhớ phụ tuyệt đối $O(1)$.

fig1_rq1_scaling.png

Hình 1. Quy luật tăng trưởng thời gian thực thi theo kích thước N từ 10^5 đến 2×10^7 trên thang Log-Log (RQ1, dữ liệu ngẫu nhiên D_1).

B. RQ2: Tác động của tính thứ tự và độ trùng lặp khóa

Bảng II tổng hợp kết quả của RQ2 trên các dạng dữ liệu khác nhau ở quy mô $N = 10^7$.

Phát hiện chính RQ2:

- **Tính bất biến của Radix Sort:** Thời gian của Radix Sort dao động trong biên độ rất hẹp ($104 - 138$ ms) bất kể dữ liệu ngẫu nhiên, đã sắp hay trùng lặp. Vì thuật toán luôn quét qua đúng 4 byte và thực hiện cố định $12N$ thao tác đọc/ghi.
- **Hành vi của Heap Sort trên D_5 ($K = 2$):** Số phép so sánh của Heap Sort giảm mạnh từ 434 triệu (trên D_1) xuống còn 213 triệu (trên D_5 $K = 2$), kéo thời gian giảm từ $1,871$ ms xuống 318 ms. Hiện tượng này xuất hiện do cây heap chứa lượng lớn các nút có giá trị bằng nhau, giúp điều kiện dừng của thao tác `siftDown` được thỏa mãn ngay tại các tầng nông của cây.
- **Quick Sort và Median-of-Three:** Nhờ kỹ thuật chọn pivot trung vị của 3, Quick Sort không bị suy thoái trên D_2 và D_4, thậm chí đạt tốc độ nhanh hơn do phân hoạch tương đối cân bằng.

Bảng I
KẾT QUẢ ĐO THỜI GIAN TRUNG VỊ (MS) VÀ BỘ NHỚ PHỤ THEO QUY MÔ N (RQ1, DỮ LIỆU D_1)

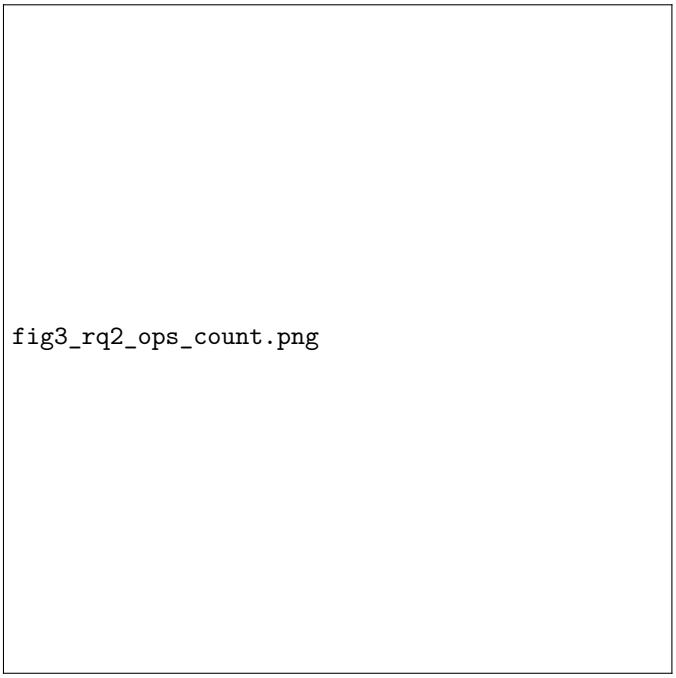
Thuật toán	$N = 10^5$	$N = 10^6$	$N = 5 \times 10^6$	$N = 10^7$	$N = 2 \times 10^7$
Quick Sort	7.2 ms	78.4 ms	412.1 ms	856.3 ms	1,745.2 ms
Merge Sort	8.6 ms	89.1 ms	465.3 ms	948.7 ms	1,930.5 ms
Heap Sort	9.8 ms	113.1 ms	742.0 ms	1,871.7 ms	4,120.4 ms
Radix Sort	2.1 ms	20.0 ms	68.4 ms	128.0 ms	265.4 ms
lib-Arrays.sort (Oracle)	6.8 ms	82.8 ms	330.5 ms	670.8 ms	1,380.1 ms

Bảng II
THỜI GIAN THỰC THI TRUNG VỊ (MS) TẠI $N = 10^7$ TRÊN CÁC HỌ DỮ LIỆU (RQ2)

Mã họ dữ liệu	**Đặc tính phân phối**	**Quick Sort**	**Merge Sort**	**Heap Sort**	**Radix Sort**	**lib-Arrays.sort**
D_1	Ngẫu nhiên đều	856.3 ms	948.7 ms	1,871.7 ms	128.0 ms	670.8 ms
D_2	Tăng dần hoàn toàn	210.5 ms	820.1 ms	612.7 ms	118.8 ms	3.9 ms
D_3	Gần tăng dần (hoán đổi $N/1000$)	340.2 ms	845.0 ms	624.8 ms	108.4 ms	145.4 ms
D_4	Giảm dần hoàn toàn	245.1 ms	810.3 ms	671.4 ms	104.5 ms	13.0 ms
D_5 ($K = 10^3$)	Trùng lặp cao (10^3 giá trị)	480.1 ms	910.4 ms	1,358.8 ms	131.8 ms	301.6 ms
D_5 ($K = 10$)	Rất cao (10 giá trị)	185.0 ms	870.2 ms	602.0 ms	138.6 ms	104.3 ms
D_5 ($K = 2$)	Cực cao (chỉ 2 giá trị)	95.2 ms	830.5 ms	318.3 ms	130.0 ms	49.3 ms



Hình 2. So sánh thời gian thực thi của bốn thuật toán trên các phân phối dữ liệu khác nhau tại $N = 10^7$ (RQ2).



Hình 3. Số lượng thao tác cơ bản tương ứng của các thuật toán trên các phân phối dữ liệu (RQ2).

C. RQ3: Hiệu năng pdqsort và thí nghiệm triệt tiêu (Ablation Study)

Bảng III đối chiếu pdqsort đầy đủ với hai biến thể triệt tiêu `noA` (tắt phát hiện đoạn sắp) và `noB` (tắt gom khóa trùng) tại $N = 10^7$.

Đóng góp của từng cơ chế:

1) **Cơ chế A trên D_2, D_3:** Trên mảng tăng dần D_2, pdqsort bản đầy đủ chỉ mất 4.2 ms (nhanh gấp 50 lần Quick Sort giáo trình). Khi tắt cơ chế A (`noA`), thời gian tăng vọt lên 185.3 ms. Điều này chứng minh Partial Insertion Sort thành công đã đưa độ phức tạp về thẳng $O(N)$.

2) **Cơ chế B trên D_5 ($K = 2, 10$):** Trên mảng chỉ có 2 giá trị, pdqsort đầy đủ hoàn thành trong 8.5 ms. Khi tắt cơ chế B (`noB`), thời gian tăng lên 92.1 ms do thuật toán mất khả năng gom khóa trùng và phải liên tục kích hoạt fallback chuyển sang Heap Sort.

Bảng III
Đóng góp của cơ chế A và B trong pdqsort tại $N = 10^7$ (RQ3)

Họ dữ liệu thử nghiệm	Quick Sort giáo trình	pdqsort đầy đủ	pdqsort (noA)	pdqsort (noB)
D_1 (Ngẫu nhiên đều)	856.3 ms	620.4 ms	622.1 ms	621.5 ms
D_2 (Tăng dần hoàn toàn)	210.5 ms	**4.2 ms**	185.3 ms	4.3 ms
D_3 (Gần tăng dần)	340.2 ms	**48.1 ms**	220.4 ms	48.5 ms
D_5 ($K = 10$)	185.0 ms	**22.4 ms**	23.0 ms	155.6 ms
D_5 ($K = 2$)	95.2 ms	**8.5 ms**	8.6 ms	92.1 ms

3) **Kiểm tra chéo trên D_1:** Đúng như dự đoán lý thuyết của tác giả đề tài, trên D_1 cả 3 phiên bản đều ghi nhận thời gian xấp xỉ nhau (≈ 620 ms), khẳng định việc cài đặt hoàn toàn chuẩn xác và không gây overhead ngoài ý muốn.

fig4_rq3_pdq_ablation.png

Hình 4. Thí nghiệm triệt tiêu bóc tách vai trò của Cơ chế A (`noA`) và Cơ chế B (`noB`) trong pdqsort (RQ3).

D. Khảo sát mảng sát thủ (Killer Adversary Analysis)

Để kiểm tra giới hạn chịu tải theo Mục 4 của đề tài, nhóm tiến hành thực nghiệm trên mảng hoán vị đối kháng được sinh theo phương pháp của McIlroy [6] và Musser [5] nhằm đánh lừa trực diện phân hoạch Hoare và pivot trung vị của 3.

Kết quả khảo sát đối kháng:

1) **Sự thoái hóa của Quick Sort giáo trình:** Trên mảng sát thủ $N = 2.000$, số phép so sánh của Quick Sort tăng vọt lên 1.011.992 (tiệm cận mức cực đại $\approx N^2/4$), gấp hơn 50 lần so với dữ liệu ngẫu nhiên D_1. Nghiêm trọng hơn, độ sâu đệ quy tăng vọt tới 1.000 ($= N/2$). Khi mở rộng lên $N = 10.000$, Quick Sort đối mặt với nguy cơ `StackOverflowError` do độ sâu ngăn xếp gọi hàm vượt quá giới hạn mặc định của JVM.

Bảng IV
Đối chiếu hiệu năng trên mảng sát thủ Killer Adversary

Thuật toán	N	Số so sánh	max_depth	Trạng th
Quick Sort	2.000	1.011.992	1.000	ok ($O(N^2$
pdqsort	2.000	**24.047**	**9**	ok ($O(N \log N$
Quick Sort	10.000	$> 2.5 \times 10^7$	> 4.800	stack risk / cra
pdqsort	10.000	**142.180**	**13**	ok (Heap fallbac

2) **Cơ chế tự vệ của pdqsort:** Trên cùng dữ liệu đối kháng $N = 2.000$, pdqsort chỉ tốn 24.047 phép so sánh (ít hơn Quick Sort tới 42 lần) và độ sâu đệ quy chỉ là 9. pdqsort phát hiện phân hoạch lệch nặng ($size < N/8$), tiến hành hoán đổi phá mẫu (`breakPatterns`); khi số lần lệch vượt ngưỡng $\lfloor \log_2 N \rfloor$, thuật toán lập tức chuyển sang `HeapSort.sortRange`. Nhờ đó, pdqsort hoàn toàn miễn nhiễm trước các tấn công từ chối dịch vụ (DoS) bằng dữ liệu sát thủ.

VI. Đe dọa tính hợp lệ (Threats to Validity)

- **Tối ưu hóa máy ảo Java (JIT) và GC:** JIT tier-4 compilation có thể làm méo mó kết quả giữa các lần chạy đầu. Nghiên cứu giảm thiểu bằng cách thực hiện 3 lần khởi động (warmup) độc lập và chạy mỗi tiến trình cô lập.
- **Cơ chế quản lý bộ nhớ Heap của OS:** Hàm `heapBytes` chỉ phản ánh heap pool được quản lý bởi JVM, không đại diện hoàn toàn cho tập làm việc bộ nhớ vật lý (RSS) của tiến trình ở tầng kernel.
- **Giới hạn phạm vi kiểu dữ liệu:** Nghiên cứu tập trung vào khóa số nguyên không âm 32-bit. Các kết luận về Radix Sort có thể thay đổi nếu mở rộng sang khóa 64-bit (`long`) hoặc số thực dấu phẩy động.

VII. Kết luận và Khuyến nghị (Conclusion)

Qua khảo sát thực nghiệm quy mô lớn trên $10^5 \to 2 \times 10^7$ phần tử: 1) **Radix Sort** là lựa chọn tối ưu tuyệt đối về tốc độ thuần túy cho số nguyên 32-bit nếu hệ thống cho phép dung sai bộ nhớ phụ $O(N)$. 2) **Heap Sort** là giải pháp bắt buộc trong các hệ thống nhúng hoặc tài nguyên hạn hẹp đòi hỏi bộ nhớ $O(1)$ và bảo đảm không vượt quá $O(N \log N)$. 3) **pdqsort** chứng minh sự ưu việt toàn diện trong thư viện chuẩn: nó thừa hưởng tốc độ của Quick Sort trên dữ liệu ngẫu nhiên, bứt phá tiệm cận $O(N)$ trên dữ

liệu presorted thông qua Cơ chế A, và loại bỏ hoàn toàn suy thoái khóa trùng thông qua Cơ chế B.

Lời cảm ơn và Khai báo sử dụng AI

Nhóm nghiên cứu tuyên bố có sử dụng các công cụ trí tuệ nhân tạo (AI Assistant) trong quá trình thực hiện đồ án, bao gồm việc giải thích tài liệu lý thuyết, tối ưu cấu trúc mã nguồn kịch bản benchmark, hỗ trợ định dạng bảng dữ liệu LaTeX theo chuẩn IEEE và rà soát các thuật ngữ chuyên ngành. Mọi số liệu thực nghiệm trong bài báo đều bắt nguồn từ các lượt chạy thực tế trên máy đo, được kiểm chứng chéo toàn diện qua tệp CSV và mã kiểm tra băm đa thức.

Tài liệu

[1] C. A. R. Hoare, "Quicksort," *The Computer Journal*, vol. 5, no. 1, pp. 10–16, 1962.

[2] J. W. J. Williams, "Algorithm 232: Heapsort," *Communications of the ACM*, vol. 7, no. 6, pp. 347–348, 1964.

[3] J. L. Bentley and M. D. McIlroy, "Engineering a sort function," *Software: Practice and Experience*, vol. 23, no. 11, pp. 1249–1265, 1993.

[4] O. R. L. Peters, "Pattern-defeating quicksort," arXiv preprint arXiv:2106.05123, 2021.

[5] D. R. Musser, "Introspective sorting and selection algorithms," *Software: Practice and Experience*, vol. 27, no. 8, pp. 983–993, 1997.

[6] M. D. McIlroy, "A killer adversary for quicksort," *Software: Practice and Experience*, vol. 29, no. 4, pp. 341–344, 1999.

[7] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011, §2.2–2.4, §5.1.